



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE CIENCIAS

Reporte. Practica-1

P R E S E N T A N

Cortes Nava José Luis 322115437

Gonzalez Gutierrez Antonio Tonatiuh 321195476

PROFESOR

Fernando Michel Tavera

ASIGNATURA

Computación Distribuida 2027-1

7 de octubre de 2026

Desarrollo general

Trabajamos sobre el simulador de eventos en C proporcionado en la práctica. La idea principal es simular una red donde cada nodo maneja su propio estado y se comunica con sus vecinos mediante mensajes. El simulador se encarga de gestionar los entregues y llamar a los handlers de recepción de cada proceso. Sobre esta base resolvimos los tres puntos: descubrir vecinos a dos saltos, reconstruir la topología completa y difundir un mensaje desde una fuente.

Ejercicio 1 — Vecinos de vecinos

En este primer ejercicio la meta es que cada nodo conozca los identificadores de los vecinos de sus vecinos. Para esto, al arrancar, cada nodo ejecuta `nodo_vecinos_iniciar` y manda un mensaje `MYNAME` con su lista de contactos a sus vecinos directos.

Al recibir este mensaje en `recibir_myname`, verificamos el tipo y guardamos los IDs en un conjunto (set) para evitar duplicados si varios vecinos nos pasan al mismo nodo. Como no hay reenvíos, el intercambio concluye tras procesar los envíos iniciales. Cabe mencionar que el conjunto resultante puede incluir al propio nodo si sus vecinos también lo tienen registrado; esto es correcto ya que solo estamos uniendo las listas recibidas. Si un nodo no tiene vecinos, su conjunto simplemente queda vacío.

Ejercicio 2 — Topología

Buscamos que cada nodo reconstruya el mapa completo de la red (todos los procesos y todos los canales), a pesar de que al inicio solo conoce a sus vecinos. lo logra con inundación.

Al iniciar, cada nodo i envía a sus vecinos un mensaje `POSITION( $i$ , vecinos_de_ $i$ )`. Cuando un nodo recibe un `POSITION( $k$ , ...)`:

- Si ya conocía al proceso k , ignora el mensaje. Este paso es muy importante para que la inundación termine y no se reenvíe de forma infinita.
- Si k es nuevo, lo marca como proceso conocido, registra los canales que el mensaje describe, y reenvía ese mismo `POSITION` a sus vecinos, excepto al que se lo envió.

Cada vez que se procesa un mensaje se comprueba si la topología ya está completa. decimos que esta completa cuando, para todo canal conocido, ambos extremos son procesos conocidos, es decir, cuando el nodo ya tiene identificados todos los procesos y las aristas de la red.

Como consideración, el registro de los canales se hace en el sentido en que cada nodo anuncia sus propias aristas; las aristas en sentido contrario quedan cubiertas cuando el nodo del otro extremo difunde su propia posición.

Ejercicio 3 — Broadcast ingenuo

Aquí buscamos difundir un mensaje desde un nodo fuente hacia toda la red conectada. En el arranque, únicamente la fuente crea el paquete **BROADCAST**, lo marca como visto y lo suelta a sus vecinos. Los demás procesos se quedan a la espera de recibirlo.

Cuando un nodo invoca **recibir_broadcast**, revisa la bandera **visto**. Si ya estaba activa, descarta el mensaje inmediatamente. Si es la primera vez que llega, guarda la información, activa la bandera y lo reenvía a todos sus vecinos (incluyendo a quien se lo mandó). Marcarlo antes de retransmitir es fundamental para cortar los ciclos de la red y evitar reenvíos infinitos. Si un nodo está desconectado de la fuente, lógicamente no recibirá el mensaje.